This short document describes the mathematics that the source code currently implements. This code effectively implements a gaussian transform from some gaussian coefficients to real space grid and vis versa. The two transforms are described by

$$V^c_{\alpha\beta\gamma} = \int V(\mathbf{r})(x-x_c)^\alpha (y-y_c)^\beta (z-z_c)^\gamma e^{-\mu_c(\mathbf{r}-\mathbf{r}_c)^2}, \tag{1}$$

and

$$V(\mathbf{r}) = \sum_{ij} \sum_{\alpha\beta\gamma} (x-x_c)^\alpha (y-y_c)^\beta (z-z_c)^\gamma e^{-\mu_c(\mathbf{r}-\mathbf{r}_c)^2}, \tag{2}$$

where $\mathbf{r}_c$ is the center of the gaussian, α, β, γ are positive integers with $\alpha+\beta+\gamma \leq l$ and l is an integer describing the angular momentum.

Eq.2 is implemented in the `collocate_original_standalone.cu` and Eq.1 in the `integrate_original_standalone.cu`. The original code implements the formulas directly and distinguish between rectangular and generic lattices (orthogonal lattice vectors vs generic lattice vectors) but we know it is possible to implement these formulas using some tensor contraction or matrix-matrix multiplications.

A second step is required to transform the coefficients $V^{ij}_{\alpha\beta\gamma}$ back to the form used by cp2k for the computations.

The integral Eq.1 converges exponentially and is effectively restricted to an integral over a sphere of given radius to reduce computation time. It is also possible to exploit the lattice properties such as orthorhombicity. When the lattice vectors are orthogonal, the integral can be expressed as a tensor reduction of the form

$$V_{\alpha\beta\gamma} = \sum_{ijk} P^i_\alpha P^j_\beta P^k_\gamma V_{ijk} E_{ijk} \tag{3}$$

where $P^i_\alpha = (x_i - x_c)^\alpha$, $P^j_\beta = (y_j - y_c)^\beta$, and $P^k_\gamma = (z_k - z_c)^\gamma$ are matrices with x_i, y_j, z_k discrete lattice coordinates and x_c, y_c, z_c the coordinates of the gaussian center. The tensor E_{ijk} is given by

$$E_{ijk} = \exp\left[-(\mathbf{r}_{ijk} - \mathbf{r}_c)^2\right].$$

We can use the properties of the exponential

$$\exp(-(a-b)^2) = \exp\left(-a^2\right) \exp\left(-b^2\right) \exp\left(2ab\right)$$

to calculate E_{ijk} efficiently provided that we express all coordinates in the lattice vector basis instead of the cartesian basis. In both cases, the number of exponentials to be computed is $n+m+o$ where n, m, o are the dimensions of the discrete cude containing the ball of radius r_{cut} for the orthorhombic case and $n(m+o)+mo$ for the general cases. It is particularly important when the implementation runs on CPU less so on GPU.

the goal is to optimize the implementations of Eq.1 and Eq.2.